

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Technology (CSE)

QUIZ 4**DURATION: 20 MINUTES****SUMMER SEMESTER, 2025-2026****FULL MARKS: 15**

SWE 4201: Object Oriented Concepts I

Programmable calculators are not allowed.

Answer **all 3 (three)** questions. Figures in the right margin indicate full marks of questions whereas corresponding CO and PO are written within parentheses.

1. What is one advantage of using an `enum` instead of plain `int` constants to represent a fixed set of values? 2
(CO2)
(PO2)

Solution:

With plain `int` constants, any number compiles and runs fine even if it doesn't correspond to a real constant – a typo or invalid value passes through silently. An `enum` restricts a variable to only the defined constants, so the compiler rejects an invalid value at compile time. This makes the code type-safe and easier to read.

2. Consider the following program and answer the questions briefly: 3 +
3 + 4
(CO2)
(PO2)

```
1 abstract class Animal {
2     abstract void eat();
3 }
4
5 class Dog extends Animal {
6     void eat() { System.out.println("Dog eating"); }
7 }
8
9 class Cat extends Animal {
10     void eat() { System.out.println("Cat eating"); }
11 }
12
13 class Cage<T extends Animal> {
14
15     private T value;
16     static int count = 0;
17
18     Cage(T value) {
19         this.value = value;
20         count++;
21     }
22
23     void feed() {
24         value.eat();
25     }
26
27     static <T extends Animal> void feedBoth(T a, T b) {
28         a.eat();
29         b.eat();
30     }
31 }
```

```

30     }
31 }
32
33 public class Main {
34     public static void main(String[] args) {
35         Cage<Dog> c1 = new Cage<>(new Dog());
36         Cage<Cat> c2 = new Cage<>(new Cat());
37
38         c1.feed();
39         Cage.feedBoth(new Dog(), new Cat());
40         System.out.println(Cage.count);
41     }
42 }

```

- What would go wrong with `feed()` if the class were declared as just `class Cage<T>` instead, with no bound at all?
- Could `feedBoth` simply reuse the class's `T` instead of redeclaring it? Explain your answer.
- Trace the program and give all lines of output.

Solution:

(a) Without a bound, `T` could be any type at all – including one with no `eat()` method. `feed()` calls `value.eat()`, which only compiles because `Animal` guarantees every possible `T` has that method. Remove the bound and `feed()` itself would fail to compile, since the compiler could no longer be sure `eat()` exists.

(b) No – `feedBoth` must declare its own `T`, and this is exactly because it is `static`. A static method can never see or use the class's own `T`, since that `T` only becomes a real type once a specific `Cage<Dog>` or similar is actually created – but `Cage.feedBoth(...)` can be called without ever creating a `Cage` at all. So `feedBoth` has nothing to borrow from and must introduce (and bound) its own placeholder.

(c) Output:

```

1 Dog eating
2 Dog eating
3 Cat eating
4 2

```

`c1.feed()` calls `eat()` on the `Dog` stored inside `c1` → "Dog eating". `Cage.feedBoth(new Dog(), new Cat())` calls `eat()` on each argument in turn → "Dog eating", then "Cat eating". `Cage.count` → 2.

`count` is 2 because `Cage<Dog>` and `Cage<Cat>` are different types only while the compiler is checking the code – at runtime, due to type erasure, both are really just the same single `Cage` class. Since `count` is `static`, it belongs to that one shared class rather than to any particular `T`, so both constructor calls (regardless of what `T` was) increment the same counter.

- Draw a rough sketch of a meme related to this course. The only wrong answer is leaving it blank.